

Task1:

7 14

Task2:

```
numbers = [10,20,30,40,50]
```

```
total = 0
```

```
for i in range(len(numbers)):
```

```
    total += numbers[i]
```

```
print(total)
```

Task3:

```
for i in range(10):
```

```
    for j in range(10):
```

```
        print(i, j)
```

a) $10 \times 10 = 100$ times

b) $1000 \times 1000 = 1,000,000$ times

c) $10 \times 1,000,000 = 10,000,000$ times

Task4:

Algorithm A scans the array once: $O(n)$

Algorithm B compares every element with every element: $O(n^2)$

Algorithm B grows faster.

When $n = 1,000,000$:

Algorithm A needs about 1,000,000 operations.

Algorithm B needs about: $1,000,000^2 = 10^{12}$ operations

So Algorithm B is extremely slow for large data.

Task5:

From fastest-growing to slowest-growing:

$2^n > n^2 > n \log_2 n > n > \log_2 n > 1$

The most problematic when n becomes large are:

1. 2^n — exponential growth.
2. n^2 — quadratic growth.

3. $n \log_2 n$ is usually acceptable and is much better than n^2 .

Task6:

[3, 8, 12, 17, 24, 31, 45, 51, 63]

1) Use linear search: $3 \rightarrow 8 \rightarrow 12 \rightarrow 17 \rightarrow 24 \rightarrow 31 \rightarrow 45$

45 is found after 7 checks.

Time complexity: $O(n)$

2) Use binary search.

Indexes 0 to 8:

1. Middle index is 4, value is 24. Since $24 < 45$, search the right half.
2. Middle index of the right half is 6, value is 45. Found.

Only about 2 comparisons are needed.

Time complexity: $O(\log n)$

Task7:

max_value = -infinity

for each number x:

if $x > \text{max_value}$:

max_value = x

output max_value

Total examinations: 500,000,000 times

Memory needed: $O(1)$

Task8:

For finding the maximum only:

- Use a single scalar variable, such as max_value.
- If processing in chunks, use a list of local maxima.

If other statistics are also needed:

- Use a histogram or counting array.
- For marks between 0 and 100, a histogram of size 101 is ideal.

Task9:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$A + B =$$

$$\begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix}$$

=

$$\begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

$$A \times B =$$

$$\begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix}$$

=

$$\begin{bmatrix} 5+14 & 6+16 \\ 15+28 & 18+32 \end{bmatrix}$$

=

$$\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Total multiplications: $4 \times 2 = 8$ multiplications

Task10:

A program scans a huge array. The CPU is very fast, but the program is still slow. Three possible reasons:

1. **Memory bandwidth or cache misses**
The array is too large for CPU cache, so the CPU waits for main memory.
2. **Disk or network I/O**
The data may be on disk, on the network, or in swap space, which is much slower than memory.
3. **The program does not use the CPU efficiently**
For example, it runs in an interpreter, uses only one thread, has dependencies, or has lock contention.

Other possible reasons include virtual memory paging, poor branch prediction, or lack of vectorization.

Task11:

`./program`

The general process is:

1. The shell parses the command and finds the executable program.
2. The shell asks the operating system to create a new process.
3. The operating system loads program into memory.
4. The process address space, stack, heap, and dynamic libraries are set up.
5. The CPU starts executing at the program entry point, usually main.
6. The program initializes variables and reads input, files, or network data.
7. The program performs computation and may make system calls.
8. Output is sent to the terminal through system calls.
9. The program finishes and the process exits.
10. The operating system reclaims resources, and the shell displays the output.

Task12:

Four programs run at the same time:

Program	Behavior	Likely bottleneck
A	Uses lots of CPU	CPU cores / computation
B	Uses lots of memory	Memory capacity / bandwidth
C	Reads a large file	Disk I/O

Program	Behavior	Likely bottleneck
D	Waits for user input	I/O latency; CPU is idle

Can all four run at the same time?

Yes, the operating system can schedule them concurrently.

However, concurrency is not always true parallelism. If there is only one CPU core, CPU-heavy program A will occupy the CPU, and the others will run by time slicing.

What determines CPU time?

- Number of CPU cores.
- Operating system scheduling policy.
- Program priority.
- Whether the program is CPU-bound or I/O-bound.
- Locking, synchronization, and memory waits.

If there is one CPU core:

- A, B, C, and D cannot truly run at the same time.
- They run by fast switching.
- When D waits for user input, the CPU can run other programs.

Task13:

A program processes 1 billion records in 10 hours. A student says: "If we buy a computer that is twice as fast, the program will take 5 hours."

Do I agree?

Not necessarily.

If the program is purely CPU-bound and has no other limits, then a 2× faster CPU could make it take about 5 hours.

But in reality, it may be limited by:

- Disk I/O.
- Memory bandwidth.
- Network latency.
- A serial part that cannot be parallelized.
- Locks and synchronization.
- An inefficient algorithm.

According to Amdahl's Law, only the part that can be sped up will improve. So it is not guaranteed to become 5 hours.

Task14:

Without changing the answer, possible ways include:

1. Use a more efficient algorithm.
2. Use parallelization: threads, processes, GPU, or distributed computing.
3. Use vectorization, such as NumPy or BLAS.
4. Cache repeated computations.
5. Reduce disk I/O and use batch reads/writes.
6. Use asynchronous I/O to overlap computation and I/O.
7. Improve memory access and cache locality.
8. Reduce dynamic memory allocation and object creation.
9. Use a faster language or compiler optimization.
10. Reduce locks and synchronization; improve load balancing.
11. Use faster hardware: SSD, more memory, faster CPU.
12. Prune or stop early if it does not change the final answer.
13. Compress data to reduce transfer size.
14. Precompute reusable parts.

Task15:

Execution times:

A = 2s

B = 2s

C = 50s

D = 3s

E = 2s

Total = 59s

If A, B, D, and E become 10× faster:

A = 0.2s

B = 0.2s

D = 0.3s

E = 0.2s

C = 50s

New total = $0.2 + 0.2 + 50 + 0.3 + 0.2 = 50.9\text{s}$

Overall speedup:

$59 / 50.9 \approx 1.16$

So the entire program becomes about 1.16× faster.

The bottleneck is C:

$50 / 59 \approx 84.7\%$

Optimization effort should go to C.

If C also becomes 10× faster:

A = 2s

B = 2s

C = 5s

D = 3s

E = 2s

New total = 14s

Speedup = $59 / 14 \approx 4.21$

Task16:

There are 10 million student marks, each between 0 and 100. We need:

- Minimum
- Maximum
- Average
- Median
- Number scoring ≥ 90
- Number scoring < 40

Algorithm

Because marks are between 0 and 100, use a histogram array of size 101:

hist[0] to hist[100]

Initialize:

min_value = 101

max_value = -1

sum = 0

count_ge_90 = 0

count_lt_40 = 0

hist[0..100] = 0

For each mark m:

sum += m

if m < min_value: min_value = m

if m > max_value: max_value = m

hist[m] += 1

if m >= 90: count_ge_90 += 1

if m < 40: count_lt_40 += 1

After scanning:

- Minimum = min_value
- Maximum = max_value
- Average = sum / 10,000,000
- Number ≥ 90 = count_ge_90
- Number < 40 = count_lt_40

Median

Total number of marks:

$N = 10,000,000$

This is even, so the median is the average of the 5,000,000th and 5,000,001st marks after sorting.

Use cumulative counts from the histogram:

cumulative = 0

for score = 0 to 100:

 cumulative += hist[score]

 if cumulative >= 5,000,000 and first candidate not found:

 first = score

 if cumulative >= 5,000,001:

 second = score

 break

median = (first + second) / 2

How many times must the data be examined?

Each mark only needs to be examined once.

Total:

10,000,000 times

The median is computed from the 101 histogram buckets, so the original data does not need to be scanned again.

Memory requirements

Only need:

- One histogram array of size 101.
- A few scalar variables.

Memory is very small: $O(1)$

If the raw data must be stored:

- 1-byte unsigned integers: about 10 MB.

- 4-byte integers: about 40 MB.
- A Python list would be much larger.

This algorithm does not need to store all raw marks.

Likely bottleneck

- Reading 10 million marks from I/O.
- If data is in memory, memory bandwidth and per-element processing speed.
- Python loops may be slow.
- Histogram updates are small and cache-friendly, so they are not the main bottleneck.

How to make it faster without changing the answer

1. Read and process in chunks.
2. Use multiple threads or processes on different chunks.
3. Each thread keeps a local histogram, sum, min, max, and counts; merge at the end.
4. Use NumPy operations such as `bincount`, `min`, `max`, and `sum`.
5. Use a binary format or memory-mapped file.
6. Use asynchronous I/O to overlap reading and computation.
7. Avoid Python per-element loops; use vectorization or a C extension.
8. Use 64-bit integers for the sum to avoid overflow.
9. Merge histograms first, then compute the median, so the answer is unchanged.